

EtherCAT[®] Modular Device Profile

Part 1: General MDP Device Model

Document: ETG.5001.1 S (D) V0.9.0

Nomenclature:	ETG-Number	ETG.5001.1
	Type	S (Standard)
	State	D (Draft)
	Version	0.9.0

Created by:	ETG
Contact:	info@ethercat.org
Date:	17.02.2016

LEGAL NOTICE

Trademarks and Patents

EtherCAT® is registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany. Other designations used in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owners.

Disclaimer

The documentation has been prepared with care. The products described are, however, constantly under development. For that reason the documentation is not in every case checked for consistency with performance data, standards or other characteristics. In the event that it contains technical or editorial errors, we retain the right to make alterations at any time and without warning. No claims for the modification of products that have already been supplied may be made on the basis of the data, diagrams and descriptions in this documentation.

Copyright

© EtherCAT Technology Group

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design.

CONTENTS

1	Scope	6
1.1	General	6
1.2	Document structure	6
2	References	7
3	Terms, Definitions and Word Usage	8
3.1	Terms and Definitions	8
3.2	Word usage: shall, should, may, can	8
4	MDP Device model	9
4.1	General model	9
4.2	Object Dictionary	11
4.2.1	EtherCAT Communication Area (0x1000...0x1FFF)	12
4.2.2	Object Areas of the Modules (0x6000...0xBFFF)	12
4.2.3	Object Areas of the Device (0xF000...0xFFFF)	14
4.3	Process Data	20
4.3.1	Output Data	20
4.3.2	Input Data	21
4.4	EtherCAT State Machine	21
4.4.1	Fieldbus Gateways	21
4.4.2	Modular Devices	22
4.4.3	Module Devices	22
4.5	AoE for acyclic access to modules	23
5	Module Object Dictionary	24
5.1	Representation of the module's objects	24
5.2	PDO Mapping and PDO Assign	25
5.2.1	Representation of the module's PDOs	25
5.2.2	PDO Mapping	27
5.2.3	PDO Assign	28

TABLES

Table 1: Device Types	9
Table 2: Module Representation	9
Table 3: Object Dictionary Structure	11
Table 4: Device Types and Object Areas	12
Table 5: Object Areas of the Device	14
Table 6: 0xF000 Module Device Profile Object	15
Table 7: Entries of 0xF000 Modular Device Profile Object	15
Table 8: 0xF00E Module PDO Group Mapping Alignment PDO Number Object	16
Table 9: Entries of 0xF00E Module PDO Group Mapping Alignment PDO Number Object	16
Table 10: 0xF00F Module PDO Group Mapping Alignment Object	16
Table 11: Entries of 0xF00F Module PDO Group Alignment Object	17
Table 12: 0xF010...0xF01F: Module Profile List Object	17
Table 13: Entries of 0xF010...0xF01F: Module Profile List Object	17
Table 14: 0xF020...0xF02F: Configured Address List Object	17
Table 15: Entries of 0xF020...0xF02F Configured Address List Object	18
Table 16: 0xF030...0xF03F: Configured Module Ident Object	18
Table 17: Entries of 0xF030...0xF03F Configured Module List Object	18
Table 18: 0xF040...0xF04F: Detected Address List Object	18
Table 19: Entries of 0xF040...0xF04F Detected Address List Object	19
Table 20: 0xF050...0xF05F Detected Module Ident Object	19
Table 21: Object 0xF050...0xF05F Detected Module List Object	19
Table 22: Object 0x1C12: RxPDO Assign	20
Table 23: Object 0x1C13: TxPDO Assign	21
Table 24: EtherCAT State Behavior for Fieldbus Gateways	21
Table 25: EtherCAT State Behavior for Modular Devices	22
Table 26: EtherCAT State Behavior for Module Devices	22
Table 27: Object 0x9nn0: Information Objects	24
Table 28: TxPDO Representation	26
Table 29: RxPDO Representation	26
Table 30: RxPDO Mapping of the device	28
Table 31: TxPDO Mapping of the device	28

FIGURES

Figure 1: Example of MDP devices	6
Figure 2: Fieldbus gateway examples	10
Figure 3: Modular device examples	10
Figure 4: Module Device examples	10
Figure 5: Model of a modular device	11
Figure 6: Multi Object Dictionary device model	24
Figure 7: Mapping of Module PDO to Device Object Dictionary	25
Figure 8: PDO Mapping of Modules to EtherCAT	27

ABBREVIATIONS

ADS	Automation Device Specification
AoE	ADS over EtherCAT
C	Conditional
CoE	CAN Application Protocol over EtherCAT
DC	Distributed Clock
DPRAM	Dual-Ported RAM
ECAT	EtherCAT
ENI	EtherCAT Network Information (EtherCAT XML Master Configuration)
ENUM	Enumeration
EoE	Ethernet over EtherCAT
ESC	EtherCAT Slave Controller
ESI	EtherCAT Slave Information (EtherCAT XML Devices Description)
ESM	EtherCAT State Machine
ETG	EtherCAT Technology Group
FMMU	Fieldbus Memory Management Unit
FoE	File Access over EtherCAT
I/O	Input/Output
IEC	International Electrotechnical Commission
LSB	Least Significant Bit
M	Mandatory
MDP	Modular Device Protocol
ms	milliseconds (10^{-3} seconds)
MSB	Most Significant Bit
ns	nanoseconds (10^{-9} seconds)
O	Optional
OD	Object Dictionary
OP	Operational
PDO	Process Data Object
PREOP	Pre-Operational
RD	Read
RO	Read-Only
ROM	Read-Only Memory
RW	Read-Write
SAFEOP	Safe-Operational
SDO	Service Data Object
SI	Subindex
SM	Sync Manager
SU	Sync Unit
WD	Watchdog
WR	Write
XML	eXtensible Markup Language

1 Scope

1.1 General

The Modular Device Profile (MDP) defines a modeling of structures within in a device. Mainly the object dictionary structure and corresponding behaviour of the entries is defined by the MDP. The intention is to provide an easy way for master and configuration tools to handle the devices.

A modular structure can be used for all kind of devices that supports physical or logical modules. This can be for example:

- Multi Axis servo drives (physical modules) with each axis having independent operation modes (logic modules)
- Extendable bus coupler with an internal backbone (e.g. sliced I/Os)
- Gateways from Fieldbusses to EtherCAT

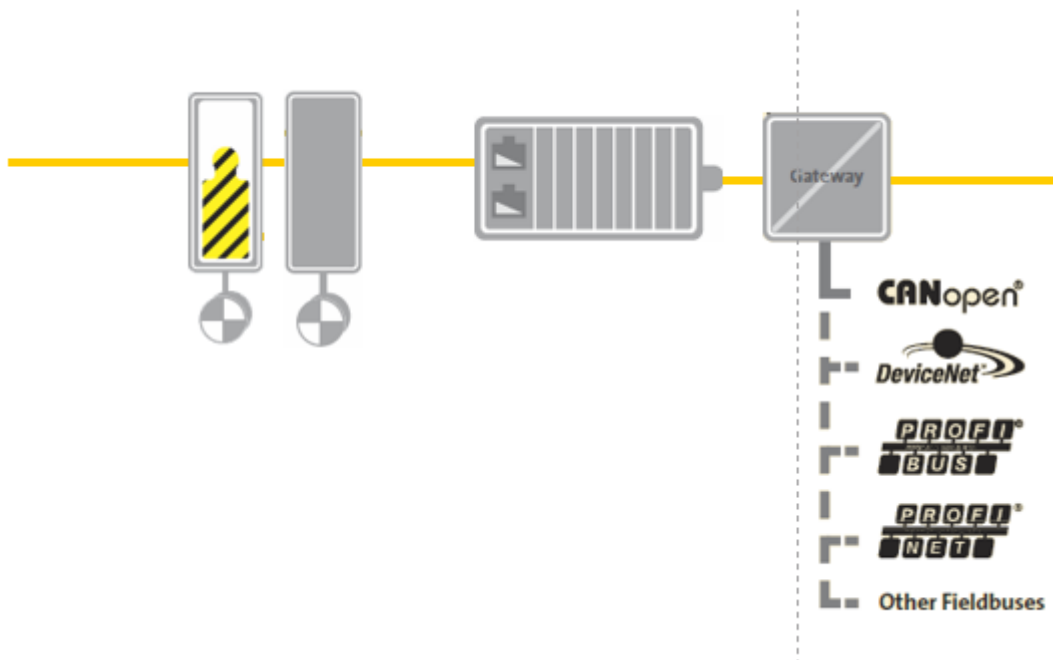


Figure 1: Example of MDP devices

The Device Description for Devices according to the MDP Profile is defined in [7].

1.2 Document structure

The MDP specification is divided in the following subparts:

- ETG.5001.1: General MDP Device Model
- ETG.5001.x: Module Profile Specification (x represents the Module Profile number)
- ETG.5001.4: MDP Safety Module Specification

2 References

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ETG Standards

- [1] ETG.1000.2: Physical Layer service definition and protocol specification
- [2] ETG.1000.3: Data Link Layer service definition
- [3] ETG.1000.4: Data Link Layer protocol specification
- [4] ETG.1000.5: Application Layer service definition
- [5] ETG.1000.6: Application Layer protocol specification
- [6] ETG.1020: EtherCAT Protocol Enhancements
- [7] ETG.2000: EtherCAT Slave Information
- [8] ETG.2100: EtherCAT Network Information
- [9] ETG.5001.1: Modular Device Profile – Part 1: General MDP Device Model (this document)
- [10] ETG.5001.x: Module Profile Specification (x represents the Module Profile number)
- [11] ETG.5001.4: Modular Device Profile – Part 4: MDP Safety Module Specification

Other References

- [12] IEC 61158-x-12 (all parts for type 12): Industrial communication networks – Fieldbus specifications
- [13] IEC 61784-2: Industrial communication networks – Profiles – Part 2: Additional fieldbus profiles for real-time networks based on ISO/IEC 8802-3
- [14] IEC 61800-7: Adjustable speed electrical power drives systems – Part 7: Generic interface and use of profiles for power drive systems

3 Terms, Definitions and Word Usage

3.1 Terms and Definitions

The terms and definitions of ETG.1000 series shall be fully valid, unless otherwise stated.

3.2 Word usage: shall, should, may, can

The word *shall* is used to indicate mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (*shall* equals *is required to*).

The word *should* is used to indicate that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain course of action is deprecated but not prohibited (*should* equals *is recommended that*).

The word *may* is used to indicate a course of action permissible within the limits of the standard (*may* equals *is permitted to*).

The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).

4 MDP Device model

4.1 General model

The main goals of this MDP device model are:

- modeling of structures within a device
- usable for a large number of devices from very simple one to complex sub-structured
- easy way for master and configuration devices to handle the device
- use of similar channel profiles in all device types shown below

The modular device profile can be applied for different device types. Depending on the device type the object dictionary can be static or dynamic. The dynamic object dictionary can be configured by the EtherCAT master during PRE-OP by writing the expected module configuration or can be generated automatically by the device after power-on.

Table 1: Device Types

Device Type	Description	Object Dictionary
fieldbus gateway	gateway to other fieldbuses	dynamic, configurable
modular device	device with physical connectable modules and/or several function modules	dynamic, automatic
module device	module which is connected directly to EtherCAT and consists of several channels	static

Table 2: Module Representation

Device Type	Module Representation
fieldbus gateway	each fieldbus station is represented as a module
modular device	each connectable sub-device or functionality is represented as a module
module device	each channel is represented as a module

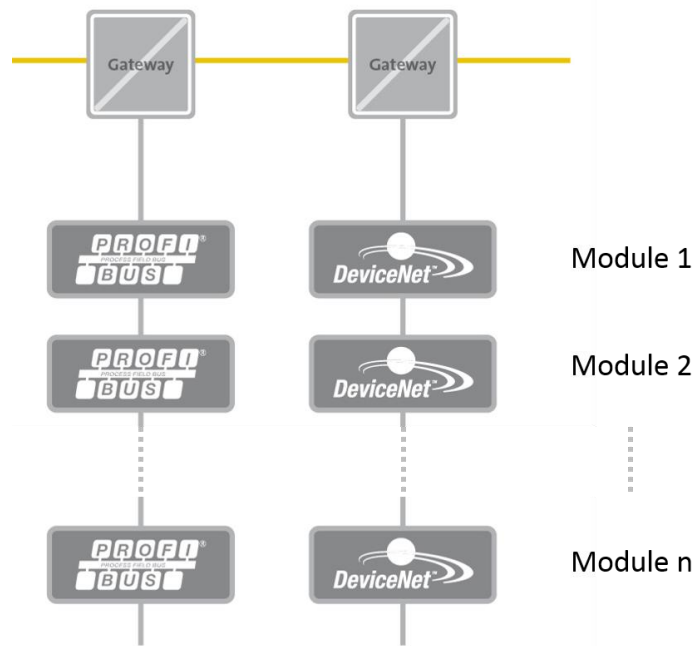


Figure 2: Fieldbus gateway examples

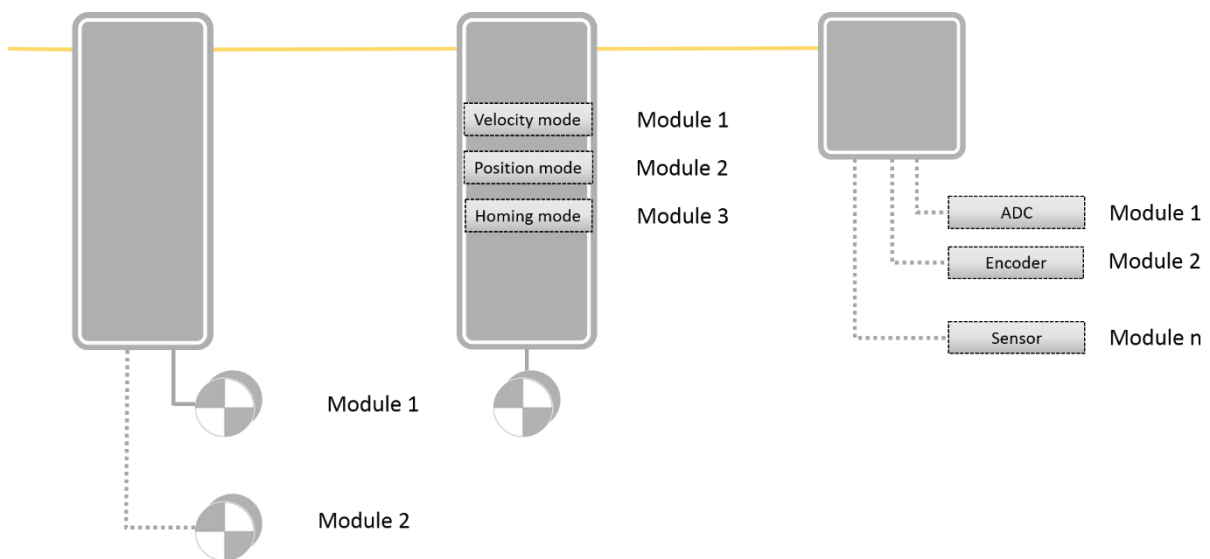


Figure 3: Modular device examples

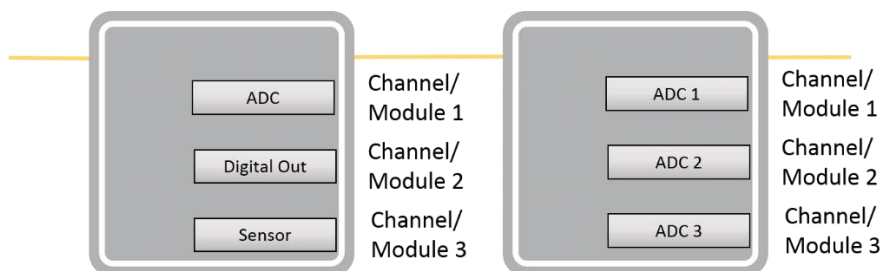


Figure 4: Module Device examples

4.2 Object Dictionary

The structure of the object dictionary for MDP devices is divided into several function specific areas.

Table 3: Object Dictionary Structure

Index	Object Dictionary Area	Description / Value
EtherCAT Communication Area , see 4.2.1		
0x1000 - 0x1FFF	Communication Area	Standard communication area
Object Area of the Modules , see 4.2.2		
0x2000 - 0x5FFF	Manufacturer Specific Area	
0x6000 - 0x6FFF	Input Area	Objects that can be mapped to TxPDOs
0x7000 - 0x7FFF	Output Area	Objects that can be mapped to RxPDOs
0x8000 - 0x8FFF	Configuration Area	Configuration and setting objects
0x9000 - 0x9FFF	Information Area	Scanned information from the modules
0xA000 - 0xAFFF	Diagnosis Area	Diagnostic, status, statistic or other information
0xB000 - 0xBFFF	Service Transfer Area	Service objects
0xC000 - 0xEFFF	Reserved Area	
Object Area of the Device , see 4.2.3		
0xF000 - 0xFFFF	Device Area	Parameters belonging to the device

Each module can occupy several objects in the function specific areas. The standard defines 16 objects per module in a specific area, but this number may be adapted to the device requirements. In the following it will be assumed that there are 16 objects per module and up to 255 modules available.

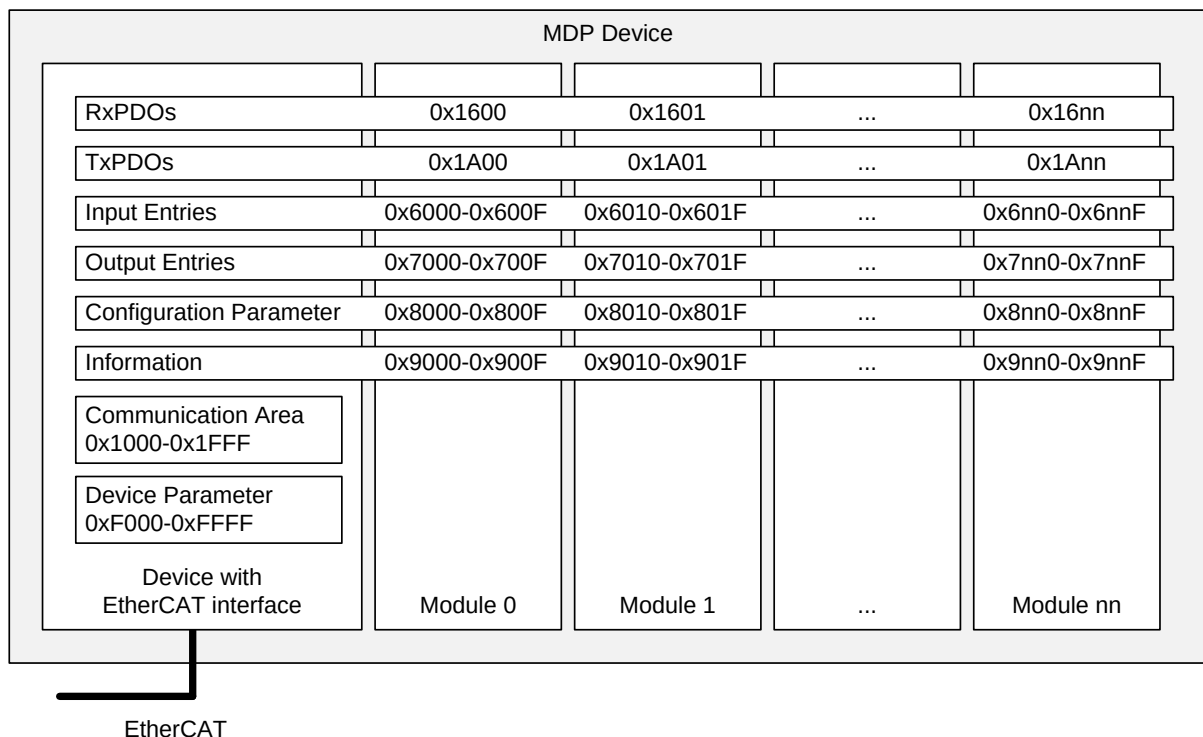


Figure 5: Model of a modular device

4.2.1 EtherCAT Communication Area (0x1000...0x1FFF)

4.2.1.1 General

The objects in the EtherCAT Communication Area are defined in [5] and [6]. MDP specific content is defined in the following sub-clauses.

4.2.1.2 Object 0x1000: Device Type

The device type object contains the device profile information.

The low-word (Bits 0-15) shall be 5001 for the modular device profile. The high-word (Bits 16-31) shall contain the module profile number defined by ETG, see [10]. If the module profile number is 0, the device supports no standardized module profile or multiple module profiles.

4.2.1.3 PDO Mapping objects (0x1600...0x17FF, 0x1A00...0x1BFF)

The PDO Mapping objects are used to describe the structure of the PDOs as described in the EtherCAT Standard [5].

The relation of the PDO Numbers to the Modules is described in the chapter Process Data, see 4.3.

4.2.1.4 PDO Assign objects (0x1C12, 0x1C13)

The PDO Assign objects are used to describe which PDOs shall be transmitted with the EtherCAT Input and Output Data.

The use of the PDO Assign for the Modular Device Profile is described in the chapter Process Data, see 4.3.

4.2.2 Object Areas of the Modules (0x6000...0xBFFF)

4.2.2.1 General

Table 4 shows which object areas are defined for the device types.

Table 4: Device Types and Object Areas

Object Area	Index Range	Fieldbus Gateway	Modular Device	Module Device
Input Data	0x6nnn	C	C	C
Output Data	0x7nnn	C	C	C
Configuration Data	0x8nnn	M	C	C
Information Data	0x9nnn	C	C	-
Diagnosis Data	0xAnnn	C	C	C
Service Transfer	0xBnnn	C	C	C

If a conditional object is mandatory is specified in the corresponding module profile.

The subindexes of these objects should follow the following rules:

- Index 0xMnn0-0xMnn6: profile specific objects per channel
- Index 0xMnn7: profile specific objects that are used only once and valid for all channels.
- Index 0xMnn8-0xMnnC: manufacturer specific objects per channel
- Index 0xMnnD: manufacturer specific objects that are used only once and valid for all channels.
- Index 0x8nnE: Vendor Data, that are used only once and valid for all channels
- Index 0x8nnF: Vendor Data per channel

With M = 6,7,8,9,A

Vendor Data are data, that can only be written by the vendor, e.g. during production.

For module related objects and entries the name values used in SDO Info and the ESI file shall be meaningful and related to the names used in the module profile specification.

4.2.2.2 Input Data object area (0x6000...0x6FFF)

The Input Data are located in the object area 0x6000...0x6FFF; each module may have up to 16 Input Data objects.

The Input Data objects of a module can be mapped in a TxPDO so that they may be read cyclically. There is one TxPDO per module defined; the index of this TxPDO Mapping object depends on the Module number (see Figure 5 and chapter 4.2). The device itself may have TxPDOs too; the index of these TxPDOs should be behind all TxPDOs of the modules.

The Input Data which shall be read cyclically by default are mapped in the module's TxPDO. If the module supports several combinations of Input Data objects which can be read cyclically the TxPDO shall be writable. The EtherCAT master could write the TxPDO mapping in PRE-OP state.

The Input Data shall be readable by SDO Upload services.

The Input Data area and the TxPDOs of a module only exist if Input Data of the module is available.

4.2.2.3 Output Data object area (0x7000...0x7FFF)

The Output Data are located in the object area 0x7000...0x7FFF; each module may have up to 16 Output Data objects.

The Output Data objects of a module can be mapped in a RxPDO so that they may be written cyclically. There is one RxPDO per module defined; the index of this RxPDO mapping object depends on the Module number (see Figure 5 and chapter 4.2). The device itself may have RxPDOs too; the index of these RxPDOs should be behind all RxPDOs of the modules.

The Output Data which shall be written cyclically by default are mapped in the module's RxPDO. If the module supports several combinations of Output Data objects which can be written cyclically the RxPDO shall be writable. The EtherCAT master could write the RxPDO mapping in PRE-OP state.

The Output Data shall be readable by SDO Upload services. The Output Data objects that are not mapped to RxPDOs may be writable by SDO Download services.

The Output Data area and the RxPDOs of a module only exist if Output Data of the module is available.

4.2.2.4 Configuration Data object area (0x8000...0x8FFF)

The Configuration Data are located in the object area 0x8000...0x8FFF; each module may have up to 16 Configuration Data objects.

Every variable of a Configuration Data object may be a setting variable. The EtherCAT master should write setting variables every time before making the state transition from PRE-OP to SAFE-OP to allow automatic device replacement.

Setting variables should be as such in the device description file ESI, see [7].

4.2.2.5 Information Data object area (0x9000...0x9FFF)

The Information Data area only exists for fieldbus gateways and modular devices (devices with dynamic object dictionary). The Information Data objects contain the information scanned from the module(s). The Information Data objects are located in the object area 0x9000...0x9FFF; each module may have up to 16 Information Data objects.

For modular devices the Information Data objects exist after power-up. It may be reinitialized after an internal communication error.

For fieldbus gateways the Information Data objects may only exist, if a Detect Modules command (0xF002) was requested by the EtherCAT master.

4.2.2.6 Diagnosis Data object area (0xA000...0xFFFF)

The Diagnosis Data are a set of diagnostic, status, statistic or other information which is readable by the EtherCAT master. The Diagnosis Data objects are located in the object area 0xA000...0xFFFF; each module may have up to 16 Diagnosis Data objects.

4.2.3 Object Areas of the Device (0xF000...0xFFFF)

4.2.3.1 General

The Device areas contain all parameters that belong to the device as shown in Table 5.

Table 5: Object Areas of the Device

Index	Name	Fieldbus Gateway	Modular Device	Module Device
0xF000	Modular Device Profile	M	M	M
0xF002	Detect Modules Command	C	C	-
0xF00E	Module PDO Group Mapping Alignment PDO Number	C	C	-
0xF00F	Module PDO Group Mapping Alignment	C	C	-
0xF010...0xF01F	Module Profile List Profile information of the modules	C	C	C M if modules have different profiles
0xF020...0xF02F	Address list of the configured Modules	M	C	-
0xF030...0xF03F	Module Ident list of the configured Modules	C	M	-
0xF040...0xF04F	Address list of detected Modules	C M if 0xF002 is supported	C	-
0xF050...0xF05F	Module Ident list of detected Modules	C	M	-
0xF100...0xF10F	Device Status (TxPDO mappable)	C	C	C
0xF110...0xF11F	Device Diagnosis	C	C	C
0xF120...0xF12F	Device Statistics	C	C	C
0xF200...0xF20F	Device Control (RxPDO mappable)	C	C	C
0xF600...0xF6FF	Input Area of device itself (TxPDO mappable)	C	C	C
0xF700...0xF7FF	Output Area of device itself (RxPDO mappable)	C	C	C
0xF800...0xF8FF	Device Configuration Parameter Area	C	C	C
0xFB00...0xFBFF	Device Command	C	C	C

If a conditional object is mandatory is specified in the corresponding module profile.

4.2.3.2 Object 0xF000: Modular Device Profile

The Modular Device Profile object contains all information to interpret the objects in function areas of the Modules. The object is read-only and not PDO mappable.

Table 6: 0xF000 Module Device Profile Object

Attribute	Value
Index	0xF000
Name	Module Device Profile
Object Code	RECORD

NOTE: The object name may be different in former slave implementations.

Table 7: Entries of 0xF000 Modular Device Profile Object

Sub-index	Name	Data Type	Access	Use	Values / Description
1	Index distance	UNSIGNED16	ro	M	Index distance between two modules (maximum number of objects per module and area) Default: 16
2	Maximum number of modules	UNSIGNED16	ro	M	Maximum Number of Modules
3	General configuration	UNSIGNED32	ro	O	Available Subindexes in General Configuration objects 0x8nn0 (Module Identification) as array of bits Bit 0 = 1 means 0x8nn0:Subindex 1 is available Bit 1 = 1 means 0x8nn0:Subindex 2 is available ... Bit 31 = 1 means 0x8nn0:Subindex 32 is available This subindex only exists for dynamic object dictionary and if the object 0x8nn0 is supported
4	General information	UNSIGNED32	ro	O	Available Subindexes in General Information objects 0x9nn0 (Module Identification) as array of bits Bit 0 = 1 means 0x9nn0:Subindex 1 is available Bit 1 = 1 means 0x9nn0:Subindex 2 is available ... Bit 31 = 1 means 0x9nn0:Subindex 32 is available This subindex only exists for dynamic object dictionary and if the object 0x9nn0 is supported
5	Module PDO Group of device	UNSIGNED16	ro	C	This subindex only exists if the modular device supports the Module PDO Groups.

NOTE: The entry names may be different in former slave implementations.

4.2.3.3 Object 0xF002: Detect Modules Command

The Detect Modules Command is defined as Command Object (see [6]), the request and response parameter depend on the device or profile. This object should be used for fieldbus master gateways to scan the fieldbus for connected slaves.

The request data contain information about the fieldbus parameters; the response data contain information about the detected fieldbus slaves.

The objects 0xF04y, 0xF05y and 0x9yyy should be updated when the Detect Modules Command is executed.

4.2.3.4 Object 0xF00E: Module PDO Group Mapping Alignment PDO Number

The Module PDO Group Mapping Alignment PDO Number object contains the PDOs used for the mapping alignment of the module groups described in chapter 4.2.3.5. If no module group needs a mapping alignment this object shall not exist.

Table 8: 0xF00E Module PDO Group Mapping Alignment PDO Number Object

Attribute	Value
Index	0xF00E
Name	Module PDO Group Mapping Alignment PDO Number
Object Code	ARRAY
Access	ro

NOTE: The object name may be different in former slave implementations.

Table 9: Entries of 0xF00E Module PDO Group Mapping Alignment PDO Number Object

Subindex	Data Type	Description / Value
1	UNSIGNED16	PDO-Indexnumber of the PDO containing the mapping alignment for Module Group 0, if Module Group 0 does not support a mapping alignment this value shall be zero
2	UNSIGNED16	PDO-Indexnumber of the PDO containing the mapping alignment for Module Group 1, if Module Group 1 does not support a mapping alignment this value shall be zero
...		...
255	UNSIGNED16	PDO-Indexnumber of the PDO containing the mapping alignment for Module Group 254, if Module Group 254 does not support a mapping alignment this value shall be zero

4.2.3.5 Object 0xF00F: Module PDO Group Mapping Alignment

The Module PDO Group Alignment object contains the alignment information for every Module PDO Group. This object is mandatory if Module PDO Groups are supported. An entry defines the alignment at the end of the Input or Output Data of the Module PDO Group.

Table 10: 0xF00F Module PDO Group Mapping Alignment Object

Attribute	Value
Index	0xF00F
Name	Module PDO Group Mapping Alignment
Object Code	ARRAY
Access	ro

NOTE: The object name may be different in former slave implementations.

Table 11: Entries of 0xF00F Module PDO Group Alignment Object

Subindex	Data Type	Description / Value
1	UNSIGNED8	Alignment in bytes at the end of the Input or Output Data of Module Group 0, if Module Group 0 does not support a mapping alignment this value shall be zero
2	UNSIGNED8	Alignment in bytes at the end of the Input or Output Data of Module Group 1, if Module Group 1 does not support a mapping alignment this value shall be zero
...		...
255	UNSIGNED8	Alignment in bytes at the end of the Input or Output Data of Module Group 254, if Module Group 254 does not support a mapping alignment this value shall be zero

4.2.3.6 Module Profile List (0xF010...0xF01F)

The Module Profile List contains the profile information of each module.

Table 12: 0xF010...0xF01F: Module Profile List Object

Attribute	Value
Index	0xF010 ... 0xF01F
Name	Module Profile List
Object Code	ARRAY
Access	ro

NOTE: The object name, write access and object code may be different in former slave implementations.

Table 13: Entries of 0xF010...0xF01F: Module Profile List Object

Subindex	Data Type	Description / Value
1	UNSIGNED32	Bit 0..15: Module profile information of the module on position 1 Bit 16...31: Module profile specific
2	UNSIGNED32	Bit 0..15: Module profile information of the module on position 2 Bit 16...31: Module profile specific
...		...
255	UNSIGNED32	Bit 0..15: Module profile information of the module on position 255 Bit 16...31: Module profile specific

If the device supports complete access, this object shall be readable by complete access.

4.2.3.7 Configured Address List (0xF020...0xF02F)

The Configured Address List is mandatory for fieldbus master gateways and shall contain the individual fieldbus specific address of the fieldbus slaves configured by the EtherCAT master as shown in Table 14 and Table 15. The Data Type is profile specific.

Table 14: 0xF020...0xF02F: Configured Address List Object

Attribute	Value
Index	0xF020 ... 0xF02F
Name	Configured Address List
Object Code	ARRAY

Attribute	Value
Access	r(w)*

NOTE: The object name may be different in former slave implementations.

*: The object/entries may only be accessible in PreOP.

Table 15: Entries of 0xF020...0xF02F Configured Address List Object

Subindex	Description / Value
1	Address of the configured module 1
2	Address of the configured module 2
...	...
255	Address of the configured module 255

4.2.3.8 Configured Module Ident List (0xF030...0xF03F)

The Configured Module Ident List contains the module ident numbers of the configured modules as shown in Table 16 and Table 17. The slave may indicate in the ESI file that the EtherCAT Master should download the Module Ident List in the transition PRE-OP to SAFE-OP (see [7]).

Table 16: 0xF030...0xF03F: Configured Module Ident Object

Attribute	Value
Index	0xF030 ... 0xF03F
Name	Configured Module Ident List
Object Code	ARRAY
Access	r(w)*

NOTE: The object name may be different in former slave implementations.

*: The entries may only be writeable in PreOP. Subindex 0 may be readonly.

Table 17: Entries of 0xF030...0xF03F Configured Module List Object

Subindex	Data Type	Access	Description / Value
1	UNSIGNED32	rw	Configured module ident of the module 1
2	UNSIGNED32	rw	Configured module ident of the module 2
...	...	...	...
255	UNSIGNED32	rw	Configured module ident of the module 255

If the device supports complete access, this object shall be readable by complete access.

4.2.3.9 Detected Address List (0xF040...0xF04F)

The Detected Address List shall contain the addresses as shown in Table 18 and Table 19 of the detected modules during power-up or when the Detect Modules Command (see chapter 0) was executed. The Data Type is profile specific.

Table 18: 0xF040...0xF04F: Detected Address List Object

Attribute	Value
Index	0xF040 ... 0xF04F
Name	Detected Address List
Object Code	ARRAY/RECORD

Attribute	Value
Access	ro

NOTE: The object name may be different in former slave implementations.

Table 19: Entries of 0xF040...0xF04F Detected Address List Object

Subindex	Description / Value
1	Address of the module detected on position 1
2	Address of the module detected on position 2
...	...
255	Address of the module detected on position 255

4.2.3.10 Detected Module Ident List (0xF050...0xF05F)

The Detected Module Ident List contains the module ident numbers as shown in Table 20 and Table 21 during power-up or when the Detect Modules Command was executed.

Table 20: 0xF050...0xF05F Detected Module Ident Object

Attribute	Value
Index	0xF050 ... 0xF05F
Name	Detected Module Ident List
Object Code	ARRAY
Access	ro

NOTE: The object name may be different in former slave implementations.

Table 21: Object 0xF050...0xF05F Detected Module List Object

Subindex	Data Types	Description / Value
1	UNSIGNED32	Module Ident of the module detected on position 1
2	UNSIGNED32	Module Ident of the module detected on position 2
...		...
255	UNSIGNED32	Module Ident of the module detected on position 255

4.2.3.11 Device Status (0xF100...0xF10F)

The Device Status object shall contain status variables of the device or the communication state of the connected modules. This object can be mapped in a TxPDO to be read cyclically by the EtherCAT master.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.12 Device Diagnosis (0xF110...0xF11F)

The Device Diagnosis object shall contain diagnosis variables of the device.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.13 Device Statistics (0xF120...0xF12F)

The Device Statistics object shall contain statistic variables of the device.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.14 Device Control (0xF200...0xF20F)

The Device Control object shall contain control variables of the device. This object can be mapped in an RxPDO.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.15 Device Input Data (0xF600...0xF6FF)

The Device Input Data objects shall contain input variables of the device itself. These objects can be mapped in a TxPDO.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.16 Device Output Data (0xF700...0xF7FF)

The Device Output Data objects shall contain input variables of the device itself. These objects can be mapped in an RxPDO.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.17 Device Configuration Data (0xF800...0xF8FF)

The Device Configuration Data objects shall contain setting variables of the device itself.

For fieldbus gateways the first Configuration Data object shall be used to connect to the fieldbus when written by the EtherCAT master.

No generic object is defined within that range. The objects may be defined for each module profile individually.

4.2.3.18 Device Command (0xFB00...0xFBFF)

The Device Command objects are defined as Command Object (see [6]).

4.3 Process Data

4.3.1 Output Data

The RxPDOs of the modules will be assigned in the order of the modules in the RxPDO Assign Object 0x1C12 as shown in Table 22. If a module does not have an RxPDO there is no gap in the RxPDO Assign Object.

Generally the RxPDO(s) of the device are assigned after all module RxPDOs because the RxPDOs of the device have a higher index. To assign the RxPDO of the device at the beginning of the output data, the module groups of the modules shall begin with number 1. The device will have the module group 0 by default and would be assigned before all module RxPDOs in that case.

Table 22: Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	RxPDO Mapping Index of 1st Module with Outputs
2	RxPDO Mapping Index of 2nd Module with Outputs
...	
n	RxPDO Mapping Index of last Module with Outputs

Subindex	Assigned RxPDO
n+1	optional: 1st RxPDO Mapping Index of the device
n+2	optional: 2nd RxPDO Mapping Index of the device
...	...

4.3.2 Input Data

The TxPDOs of the modules will be assigned in the order of the modules in the TxPDO Assign Object 0x1C13 as shown in Table 23. If a module does not have a TxPDO there is no gap in the TxPDO Assign Object.

Generally the TxPDO(s) of the device are assigned after all module TxPDOs because the TxPDO of the device have a higher index. To assign the TxPDOs of the device at the beginning of the input data, the module groups of the modules shall begin with number 1. The device will have the module group 0 by default and would be assigned before all module TxPDOs in that case.

Table 23: Object 0x1C13: TxPDO Assign

Subindex	Assigned RxPDO
1	TxPDO Mapping Index of 1st Module with Inputs
2	TxPDO Mapping Index of 2nd Module with Inputs
...	
n	TxPDO Mapping Index of last Module with Inputs
n+1	optional: 1st TxPDO Mapping Index of the device
n+2	optional: 2nd TxPDO Mapping Index of the device
...	...

4.4 EtherCAT State Machine

Next to the behavior in the different EtherCAT states defined in the EtherCAT standard, [3], a more detailed behavior and allowed operations are described in this chapter.

4.4.1 Fieldbus Gateways

Table 24 shows the behavior of fieldbus gateways in the different EtherCAT states.

Table 24: EtherCAT State Behavior for Fieldbus Gateways

EtherCAT State	Behavior / Allowed Operations
INIT	not connected to the fieldbus if backup parameter were stored, loading the stored configuration from non-volatile memory

EtherCAT State	Behavior / Allowed Operations
PRE-OP	connect to the fieldbus when executing the Detect Modules command or writing the Master Configuration object or if a stored configuration was loaded from non-volatile memory creating of expected fieldbus slaves optional: download of PDO Mapping optional: download of PDO Assign detect the connected fieldbus slaves and fill the information data objects if Detect Modules command is executed
SAFE-OP	cyclic data transfer with fieldbus slaves outputs are in the safe state acyclic access to fieldbus slaves with AoE
OP	cyclic data transfer with fieldbus slaves outputs have process values acyclic access to fieldbus slaves with AoE

4.4.2 Modular Devices

Table 25 shows the behavior of modular devices in the different EtherCAT states.

Table 25: EtherCAT State Behavior for Modular Devices

EtherCAT State	Behavior / Allowed Operations
INIT	connected to the internal bus
PRE-OP	connected to the internal bus optional: download the expected module ident list optional: download of PDO Mapping optional: download of PDO Assign acyclic access to modules with AoE
SAFE-OP	cyclic data transfer with modules outputs are in the safe state acyclic access to modules with AoE
OP	cyclic data transfer with modules outputs have process values acyclic access to modules with AoE

4.4.3 Module Devices

Table 26 shows the behavior of module devices in the different EtherCAT states.

Table 26: EtherCAT State Behavior for Module Devices

EtherCAT State	Behavior / Allowed Operations
INIT	-
PRE-OP	optional: download configuration data optional: download of PDO Mapping optional: download of PDO Assign
SAFE-OP	read local inputs and write local outputs outputs are in the safe state
OP	read local inputs and write local outputs outputs have process values

4.5 AoE for acyclic access to modules

For a good performance parallel services are needed for acyclic service requests to different fieldbus slaves. AoE services should be used for accessing fieldbus slaves (or modules) with acyclic services, see [6]. AoE services can be transmitted in parallel because every AoE command contains an Invoke ID which identifies the request.

The device gets an AoE Net ID through the AoE Init Command in PRE-OP state; every module has its own Port Number. The use of the Port Number, Index Group and Index Offset depends on the module profile.

5 Module Object Dictionary

The following device types may need an extra definition for handling a multi object dictionaries device model:

- Devices with different tasks and functionality. The device have a lot of data that do not fit in a single object dictionary
- Modular devices with modules which each having its own object dictionary (e.g. that the DS4xx profiles may be supported on a module)
- Fieldbus gateways with each fieldbus slave is represented by an object dictionary

Figure 6 shows the device model.

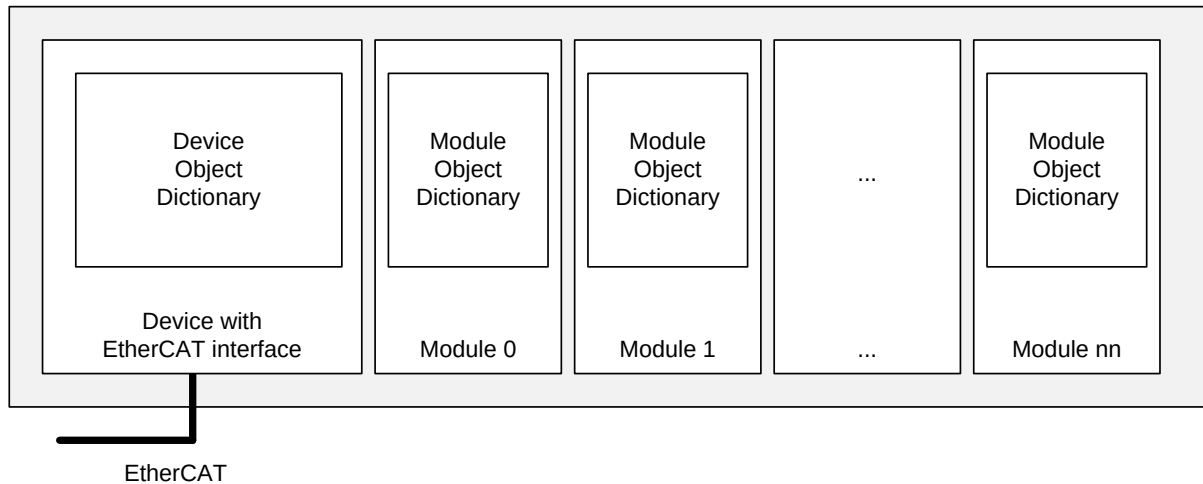


Figure 6: Multi Object Dictionary device model

The object dictionary of the device shall be structured according to the Modular Device Profile defined in clause 4. The PDO mapping and PDO assign of the device's object dictionary shall describe the EtherCAT process data.

The PDOs of the module's object dictionaries shall be represented by objects in the device's object dictionary.

So the device's object dictionary is compatible to modular devices with a single object dictionary that all EtherCAT master can communicate with Multi Object dictionary devices.

If the EtherCAT master supports AoE services additional information like the module's profile, the module's PDO mapping or the module's parameter can be accessed.

5.1 Representation of the module's objects

Some objects of the modules are represented in the device's object dictionary to get the information of the connected by reading the device's object dictionary (see Table 27).

Table 27: Object 0x9nn0: Information Objects

Index	Subindex	Name	Data Type	Description / Value
0x9000	4	Device Type	UINT32	Represents the Device type (Object 0x1000) of the first module
	5	Vendor ID	UINT32	Represents the Vendor ID (Entry 0x1018:01) of the first module
	6	Product Code	UINT32	Represents the Product Code (Entry 0x1018:02) of the first module
	7	Revision	UINT32	Represents the Revision (Entry 0x1018:03) of the first module

Index	Subindex	Name	Data Type	Description / Value
	8	Serial Number	UINT32	Represents the Serial Number (Entry 0x1018:04) of the first module
0x9010	4	Device Type	UINT32	Represents the Device type (Object 0x1000) of the second module
	5	Vendor ID	UINT32	Represents the Vendor ID (Entry 0x1018:01) of the second module
	6	Product Code	UINT32	Represents the Product Code (Entry 0x1018:02) of the second module
	7	Revision	UINT32	Represents the Revision (Entry 0x1018:03) of the second module
	8	Serial Number	UINT32	Represents the Serial Number (Entry 0x1018:04) of the second module
...		...		

5.2 PDO Mapping and PDO Assign

5.2.1 Representation of the module's PDOs

The TxPDOs and RxPDOs of the Modules are mapped to the object dictionary of the device. The PDOs of the modules shall be represented by the objects 0x6000...0x6FFF and 0x7000...0x7FFF in the device's object dictionary. This is shown in Figure 7.

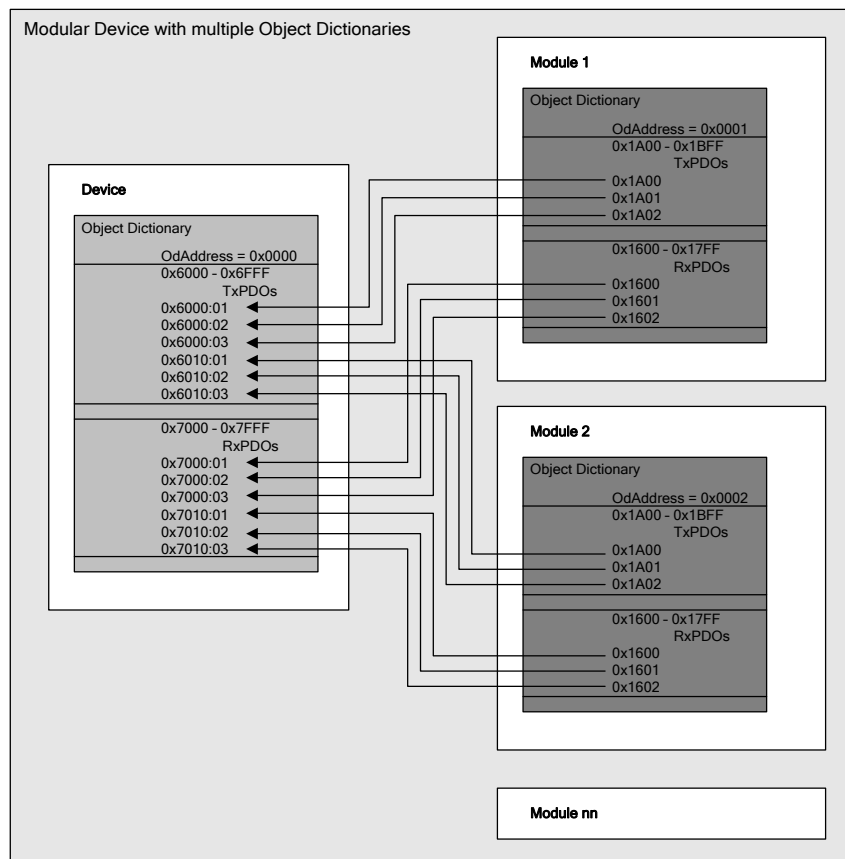


Figure 7: Mapping of Module PDO to Device Object Dictionary

Table 28 describes the TxPDOs representation of a module in the object dictionary of the device.

Table 28: TxPDO Representation

Index	Subindex	Name	Data Type	Description / Value
0x6000	1	TxPDO 1 Module 1	OCTET-STRING	Represents TxPDO 1 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1A00
	2	TxPDO 2 Module 1	OCTET-STRING	Represents TxPDO 2 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1A01
..		..		
	255	TxPDO 255 Module 1	OCTET-STRING	Represents TxPDO 255 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1AFE
0x6001	1	TxPDO 256 Module 1	OCTET-STRING	Represents TxPDO 256 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1AFF
..		...		
	255	TxPDO 510 Module 1	OCTET-STRING	Represents TxPDO 510 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1BFD
0x6002	1	TxPDO 511 Module 1	OCTET-STRING	Represents TxPDO 511 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1BFE
	2	TxPDO 512 Module 1	OCTET-STRING	Represents TxPDO 512 of the first module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1BFF
0x6010	1	TxPDO 1 Module 2	OCTET-STRING	Represents TxPDO 1 of the second module, the TxPDO mapping is defined in the object dictionary of the first module at index 0x1A00
...		...		

Table 29 describes the RxPDOs representation of a module in the object dictionary of the device.

Table 29: RxPDO Representation

Index	Subindex	Name	Data Type	Description / Value
0x7000	1	RxPDO 1 Module 1	OCTET-STRING	Represents RxPDO 1 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x1600
	2	RxPDO 2 Module 1	OCTET-STRING	Represents RxPDO 2 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x1601
..		..		
	255	RxPDO 255 Module 1	OCTET-STRING	Represents RxPDO 255 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x16FE
0x7001	1	RxPDO 256 Module 1	OCTET-STRING	Represents RxPDO 256 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x16FF
..		..		

Index	Subindex	Name	Data Type	Description / Value
	255	RxPDO 510 Module 1	OCTET-STRING	Represents RxPDO 510 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x17FD
0x7002	1	RxPDO 511 Module 1	OCTET-STRING	Represents RxPDO 511 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x17FE
	2	RxPDO 512 Module 1	OCTET-STRING	Represents RxPDO 512 of the first module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x17FF
0x7010	1	RxPDO 1 Module 2	OCTET-STRING	Represents RxPDO 1 of the second module, the RxPDO mapping is defined in the object dictionary of the first module at index 0x1600
...		...		

5.2.2 PDO Mapping

For every module one RxPDO and one TxPDO mapping object should be defined in the device's object dictionary as described in the Modular Device Profile.

The RxPDO mapping entries shall contain the index 0x7yyy, the TxPDO mapping entries shall contain the index 0x6yyy. This is shown by Figure 8.

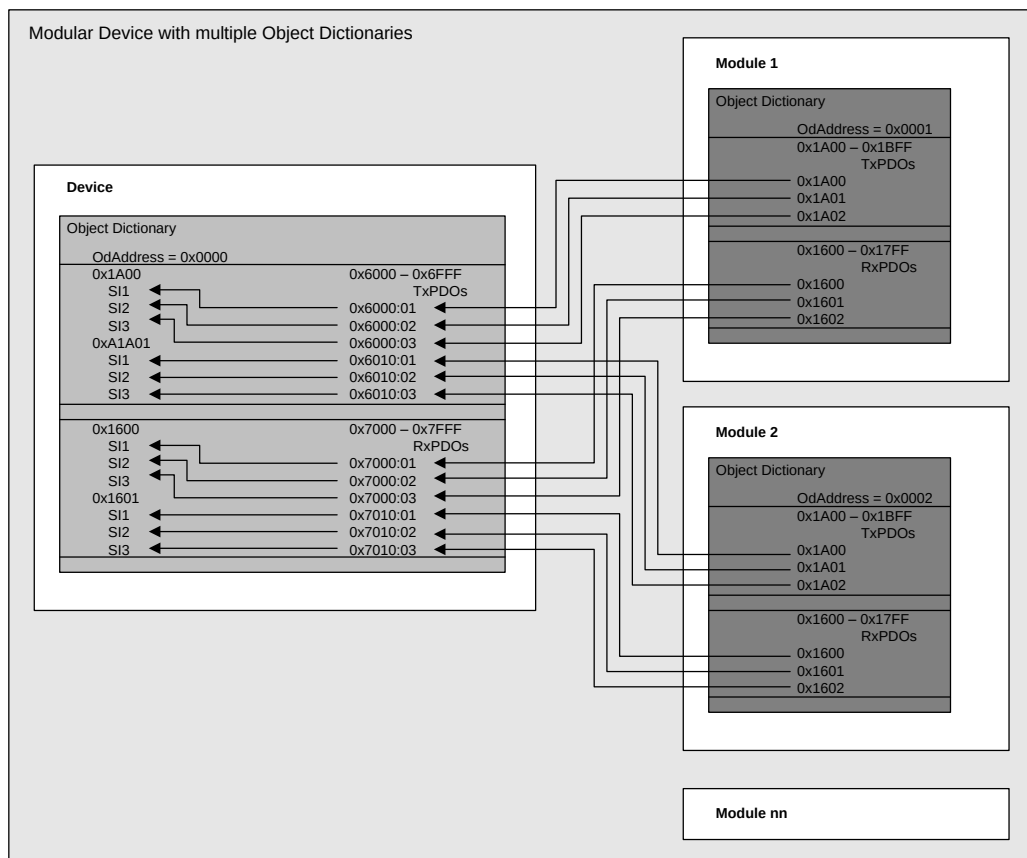


Figure 8: PDO Mapping of Modules to EtherCAT

Table 30 shows the RxPDO Mapping of the device.

Table 30: RxPDO Mapping of the device

Index	Name	Description / Value
0x1600	Module 1 RxPDO mapping	RxPDO 1, output mapping of the first module
0x1601	Module 2 RxPDO mapping	RxPDO 2, output mapping of the second module
..	...	...

Table 31 shows the TxPDO Mapping of the device.

Table 31: TxPDO Mapping of the device

Index	Name	Description / Value
0x1A00	Module 1 TxPDO mapping	TxPDO 1, input mapping of the first module
0x1A01	Module 2 TxPDO mapping	TxPDO 2, input mapping of the second module
..	...	...

If a module does not support RxPDOs or TxPDOs the corresponding object and PDO mapping indexes shall not exist in the device's object dictionary.

5.2.3 PDO Assign

The PDO assign is the same as defined in chapter 4.3.